LangChain Notes

Table of Contents

---

1. Introduction to LangChain

LangChain is a powerful framework used to build applications powered by Large Language Models (LLMs).

It helps developers connect LLMs with:

- External tools
- APIs
- Databases
- Vector stores
- Memory
- Agents
- RAG systems

LangChain simplifies the development of AI applications such as:

- Chatbots
- AI assistants
- RAG systems
- AI agents
- Document Q&A systems
- AI workflows

---

2. Why LangChain?

Without LangChain:

- Developers manually manage prompts
- Difficult to integrate memory
- Hard to connect tools
- Complex multi-step AI workflows

With LangChain:

- Easy chaining
- Better prompt management
- Supports RAG
- Supports agents
- Supports memory
- Supports multiple LLM providers

---

3. LangChain Architecture

Core Components

1. Models
2. Prompts

3. Chains
4. Memory
5. Retrievers
6. Agents
7. Tools
8. Output Parsers

---

4. Installing LangChain

Install Required Packages

```
pip install langchain
pip install langchain-community
pip install langchain-openai
pip install langchain-google-genai
pip install faiss-cpu
pip install chromadb
pip install pypdf
pip install tiktoken
```

---

5. LLMs in LangChain

LLM stands for Large Language Model.

LLMs generate text based on prompts.

Examples:

- GPT Models
- Gemini
- Claude
- Llama
- Mistral

Using OpenAI LLM

```
from langchain_openai import OpenAI
```

```
llm = OpenAI(
    api_key="YOUR_API_KEY",
    temperature=0.7
)

response = llm.invoke("What is Machine Learning?")

print(response)
```

Parameters of LLM

Parameter	Description
temperature	Creativity level
max_tokens	Maximum output length
top_p	Sampling control
frequency_penalty	Reduces repetition

---

6. Chat Models in LangChain

Chat Models work using messages.

Examples:

- Human Messages
- AI Messages
- System Messages

Using ChatOpenAI

Important ChatOpenAI Parameters

Parameter	Purpose
model	Specifies the model name

Parameter	Purpose
temperature	Controls creativity
max_tokens	Maximum response length
top_p	Controls token sampling
frequency_penalty	Reduces repeated words
presence_penalty	Encourages new topics
api_key	Authentication key
streaming	Enables streaming responses

Parameter Explanation

temperature

Controls randomness.

Value	Behavior
0.0	Deterministic output
0.5	Balanced output
1.0	Creative output

---

max_tokens

Controls maximum response length.

Example:

- Small response = 100
- Long article = 1000+

---

top_p

Controls diversity of generated text.

Lower value = focused output.

Higher value = diverse output.

```python
from langchain_openai import ChatOpenAI

chat_model = ChatOpenAI(
    model="gpt-4o-mini",
    temperature=0.5,
    api_key="YOUR_API_KEY"
)

response = chat_model.invoke("Explain Python in simple words")

print(response.content)
```

Using Gemini Chat Model

```python
from langchain_google_genai import ChatGoogleGenerativeAI

model = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    google_api_key="YOUR_API_KEY"
)

response = model.invoke("Explain AI")

print(response.content)
```

Difference Between LLM and Chat Model

LLM	Chat Model
Text Input	Message Input
Simple Prompt	Conversation Style
No Message Roles	Uses System/Human/AI Roles

---

7. Prompt Templates

Prompt Templates allow dynamic prompt creation.

Basic Prompt Template

PromptTemplate Parameters

Parameter	Purpose
input_variables	Variables accepted by template
template	Prompt structure
partial_variables	Pre-filled variables
validate_template	Checks template validity

```
from langchain.prompts import PromptTemplate

prompt = PromptTemplate(
    input_variables=["topic"],
    template="Explain {topic} in simple words"
)

formatted_prompt = prompt.format(topic="Machine Learning")

print(formatted_prompt)
```

Multiple Variables Prompt

```
from langchain.prompts import PromptTemplate

prompt = PromptTemplate(
    input_variables=["language", "topic"],
    template="Explain {topic} using {language} examples"
)
```

```
print(
    prompt.format(
        language="Python",
        topic="Loops"
    )
)
```

---

8. Prompt Engineering

Prompt Engineering is the process of designing effective prompts.

Types of Prompting

1. Zero-Shot Prompting

```
prompt = "Translate English to Hindi: Hello"
```

---

2. One-Shot Prompting

```
prompt = """
English: Hello
Hindi: Namaste

English: Good Morning
"""
```

---

3. Few-Shot Prompting

```
prompt = """
Q: 2 + 2
A: 4

Q: 5 + 5
A: 10
```

```
Q: 10 + 10
"""
```

---

4. Chain of Thought Prompting

```
prompt = "Explain step by step how to solve 25 * 5"
```

---

9. Messages in LangChain

Messages help manage conversations.

Types of Messages

1. HumanMessage
2. AIMessage
3. SystemMessage

Example

```
from langchain_core.messages import (
    HumanMessage,
    SystemMessage
)

messages = [
    SystemMessage(content="You are a helpful AI assistant"),
    HumanMessage(content="Explain Deep Learning")
]

response = chat_model.invoke(messages)

print(response.content)
```

---

10. Output Parsers

Output parsers structure model responses.

String Output Parser

```
from langchain_core.output_parsers import StrOutputParser

parser = StrOutputParser()

result = parser.invoke("Hello")

print(result)
```

JSON Output Parser

```
from langchain.output_parsers import ResponseSchema
from langchain.output_parsers import StructuredOutputParser

schemas = [
    ResponseSchema(name="name", description="person name"),
    ResponseSchema(name="age", description="person age")
]

parser = StructuredOutputParser.from_response_schemas(schemas)

print(parser.get_format_instructions())
```

---

11. Chains in LangChain

Chains are one of the most important concepts in LangChain.

Chains connect multiple LangChain components together to create AI workflows.

A chain can combine:

- Prompt Templates

- LLMs
- Chat Models
- Output Parsers
- Memory
- Retrievers
- Tools

The output of one component becomes the input for another component.

Why Chains are Important

Chains help:

1. Automate workflows
2. Reduce repetitive coding
3. Build multi-step AI systems
4. Create structured pipelines
5. Improve scalability

Types of Chains in LangChain

1. LLM Chain
2. Sequential Chain
3. Simple Sequential Chain
4. Parallel Chain
5. Conditional Chain
6. Router Chain
7. Retrieval Chain
8. Runnable Chains
9. Transform Chains
10. Conversation Chains

Chains combine multiple components.

Basic LLM Chain

LLMChain connects:

- Prompt Template
- LLM
- Output

It is the most basic chain in LangChain.

Understanding Parameters

Parameter	Purpose
llm	Defines the language model
prompt	Defines the input prompt template
verbose	Shows internal execution steps
memory	Stores conversation history
output_key	Stores output variable name

```
from langchain.chains import LLMChain
from langchain.prompts import PromptTemplate
from langchain_openai import OpenAI

llm = OpenAI(api_key="YOUR_API_KEY")

prompt = PromptTemplate(
    input_variables=["topic"],
    template="Explain {topic}"
)

chain = LLMChain(
    llm=llm,
    prompt=prompt
)

response = chain.run("Artificial Intelligence")

print(response)
```

---

12. Sequential Chains

Sequential Chains execute chains step-by-step.

The output of one chain becomes the input for the next chain.

Why Sequential Chains are Used

Sequential chains are useful when:

- Multi-step reasoning is needed
- AI workflows depend on previous outputs
- Building content generation pipelines

Simple Sequential Chain

A Simple Sequential Chain passes a single output directly to another chain.

Example

```
from langchain.chains import LLMChain
from langchain.chains import SimpleSequentialChain
from langchain.prompts import PromptTemplate
from langchain_openai import OpenAI

llm = OpenAI(api_key="YOUR_API_KEY")

# First Chain
prompt1 = PromptTemplate(
    input_variables=['topic'],
    template='Write a short title about {topic}'
)

chain1 = LLMChain(
    llm=llm,
    prompt=prompt1
)

# Second Chain
prompt2 = PromptTemplate(
    input_variables=['text'],
    template='Write a LinkedIn post about {text}'
)

chain2 = LLMChain(
    llm=llm,
```

```
    prompt=prompt2
)

# Sequential Chain
final_chain = SimpleSequentialChain(
    chains=[chain1, chain2],
    verbose=True
)

response = final_chain.run('Artificial Intelligence')

print(response)
```

Parameters Explanation

Parameter	Purpose
chains	List of chains to execute
verbose	Displays execution details
input_variables	Input variables for chain
output_variables	Final outputs

Full Sequential Chain

Full Sequential Chains support multiple inputs and outputs.

Example

```
from langchain.chains import SequentialChain

overall_chain = SequentialChain(
    chains=[chain1, chain2],
    input_variables=['topic'],
    output_variables=['text'],
    verbose=True
)
```

Parallel Chains

Parallel Chains run multiple chains simultaneously.

This improves speed and efficiency.

Why Parallel Chains are Important

Parallel execution helps:

- Reduce latency
- Execute independent tasks simultaneously
- Improve performance

Parallel Chain Example Using RunnableParallel

```
from langchain_core.runnables import RunnableParallel
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

model = ChatOpenAI(api_key='YOUR_API_KEY')

summary_prompt = ChatPromptTemplate.from_template(
    'Summarize {topic}'
)

question_prompt = ChatPromptTemplate.from_template(
    'Generate interview questions about {topic}'
)

summary_chain = summary_prompt | model
question_chain = question_prompt | model

parallel_chain = RunnableParallel(
    summary=summary_chain,
    questions=question_chain
)

response = parallel_chain.invoke(
    {'topic': 'Machine Learning'}
)
```

```
print(response)
```

Parameters Explanation

Parameter	Purpose
RunnableParallel	Executes chains in parallel
summary	Key for summary output
questions	Key for questions output
invoke	Executes the chain

Conditional Chains

Conditional Chains execute different chains based on conditions.

Why Conditional Chains are Important

Used when:

- Different prompts are needed
- Dynamic workflows are required
- Intelligent routing is needed

Conditional Chain Example

```
from langchain_core.runnables import RunnableBranch
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

model = ChatOpenAI(api_key='YOUR_API_KEY')

technical_prompt = ChatPromptTemplate.from_template(
    'Explain technically: {query}'
)
```

```
simple_prompt = ChatPromptTemplate.from_template(
    'Explain simply: {query}'
)

technical_chain = technical_prompt | model
simple_chain = simple_prompt | model

branch = RunnableBranch(
    (
        lambda x: 'technical' in x['query'],
        technical_chain
    ),
    simple_chain
)

response = branch.invoke(
    {'query': 'technical explanation of AI'}
)

print(response)
```

Router Chains

Router Chains automatically select the best chain.

Use Cases

- Multi-domain chatbots
- AI tutors
- Customer support systems

Conversation Chains

Conversation Chains maintain conversation history.

Example

```
from langchain.chains import ConversationChain
from langchain.memory import ConversationBufferMemory
from langchain_openai import OpenAI

llm = OpenAI(api_key='YOUR_API_KEY')

memory = ConversationBufferMemory()

conversation = ConversationChain(
    llm=llm,
    memory=memory,
    verbose=True
)

response = conversation.predict(
    input='Hello'
)

print(response)
```

Retrieval Chains

Retrieval Chains combine retrievers and LLMs.

These are heavily used in RAG systems.

Example

```
from langchain.chains import RetrievalQA

qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    chain_type='stuff'
)
```

Chain Types in RetrievalQA

Chain Type	Purpose
stuff	Combines all chunks
map_reduce	Processes chunks separately
refine	Refines answers iteratively
map_rerank	Ranks chunk outputs

Runnable Chains (LCEL)

Runnable Chains are modern LangChain pipelines.

They use:

- Pipe operators
- Streaming
- Parallel execution
- Async execution

Sequential Chains pass output from one chain to another.

Example

```
from langchain.chains import SequentialChain
```

---

13. Runnable Interface

Runnable interface is the modern LangChain execution system.

Runnable Sequence

```
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

prompt = ChatPromptTemplate.from_template(
    "Explain {topic}"
)

model = ChatOpenAI(api_key="YOUR_API_KEY")

parser = StrOutputParser()

chain = prompt | model | parser

response = chain.invoke({"topic": "Neural Networks"})

print(response)
```

---

14. Memory in LangChain

Memory stores conversation history.

Conversation Buffer Memory

```
from langchain.memory import ConversationBufferMemory

memory = ConversationBufferMemory()

memory.save_context(
    {"input": "Hi"},
    {"output": "Hello"}
)

print(memory.load_memory_variables({}))
```

Types of Memory

1. Buffer Memory
2. Summary Memory
3. Token Buffer Memory
4. Window Memory

---

15. Document Loaders

Document loaders load files into LangChain.

PDF Loader

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("sample.pdf")

documents = loader.load()

print(documents)
```

Text Loader

```
from langchain_community.document_loaders import TextLoader

loader = TextLoader("sample.txt")

docs = loader.load()
```

CSV Loader

```
from langchain_community.document_loaders.csv_loader import CSVLoader

loader = CSVLoader(file_path="data.csv")
```

```
docs = loader.load()
```

---

16. Text Splitters

Text splitters divide documents into chunks.

Recursive Character Text Splitter

Text Splitters divide large documents into smaller chunks before creating embeddings.

This is extremely important in RAG systems because LLMs have token limits.

Important Parameters in Text Splitters

Parameter	Purpose
chunk_size	Maximum size of each text chunk
chunk_overlap	Overlapping text between chunks
separators	Characters used for splitting
length_function	Measures chunk length

Why chunk_size is Important

chunk_size controls how much text goes into one chunk.

Example:

- Small chunk_size = better retrieval precision
- Large chunk_size = more context

Typical values:

Use Case	chunk_size
Small PDFs	300-500

Use Case	chunk_size
RAG Systems	500-1000
Large Documents	1000-2000

Why chunk_overlap is Important

chunk_overlap preserves context between chunks.

Without overlap:

- Sentences may break
- Important context may be lost

Typical values:

chunk_size	Recommended overlap
500	50-100
1000	100-200

Example Visualization

Chunk 1: "Machine Learning is a field of AI..."

Chunk 2 with overlap: "field of AI that allows systems..."

The overlapping text helps preserve meaning.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)

chunks = splitter.split_documents(documents)
```

```
print(chunks)
```

Why Text Splitting is Important

- Better retrieval
- Faster embeddings
- Better RAG accuracy

---

17. Embeddings

Embeddings convert text into numerical vectors.

OpenAI Embeddings

Embeddings convert text into high-dimensional numerical vectors.

These vectors help AI systems understand semantic meaning.

Important Embedding Parameters

Parameter	Purpose
model	Embedding model name
api_key	Authentication key
dimensions	Embedding vector size
chunk_size	Batch processing size

Why Embeddings are Important

Embeddings help:

- Similarity Search
- Semantic Search

- RAG Systems
- Recommendation Systems
- Clustering

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(
    api_key="YOUR_API_KEY"
)
```

Gemini Embeddings

```
from langchain_google_genai import GoogleGenerativeAIEmbeddings

embeddings = GoogleGenerativeAIEmbeddings(
    model="models/embedding-001",
    google_api_key="YOUR_API_KEY"
)
```

---

18. Vector Databases

Vector databases store embeddings.

Examples:

- FAISS
- ChromaDB
- Pinecone
- Weaviate

Using FAISS

FAISS is a vector database used for fast similarity search.

Important FAISS Concepts

Concept	Purpose
Vector Store	Stores embeddings
Similarity Search	Finds related chunks
Retriever	Retrieves relevant documents
Top-k Search	Returns top matching chunks

Important Parameters

Parameter	Purpose
k	Number of chunks retrieved
search_type	Similarity search method
score_threshold	Minimum similarity score

```
from langchain_community.vectorstores import FAISS

vectorstore = FAISS.from_documents(
    chunks,
    embeddings
)
```

Similarity Search

```
results = vectorstore.similarity_search(
    "What is Machine Learning?"
)

print(results)
```

---

19. Retrieval-Augmented Generation (RAG)

RAG combines:

- Retrieval System
- LLM Generation

It allows AI to answer questions using external knowledge.

RAG Workflow

1. Load Documents
2. Split Text
3. Create Embeddings
4. Store in Vector DB
5. Retrieve Relevant Chunks
6. Send to LLM
7. Generate Final Answer

Full RAG Example

Important RAG Parameters

Parameter	Purpose
chunk_size	Size of document chunks
chunk_overlap	Preserves chunk continuity
retriever	Retrieves relevant chunks
temperature	Controls response creativity
k	Number of retrieved chunks
search_kwargs	Retriever search configuration
chain_type	Method used to combine retrieved chunks

Understanding chain_type

Chain Type	Explanation
stuff	Combines all chunks directly
map_reduce	Processes chunks separately and combines answers
refine	Improves answer iteratively
map_rerank	Ranks chunk answers

```
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_google_genai import (
    GoogleGenerativeAIEmbeddings,
    ChatGoogleGenerativeAI
)
from langchain_community.vectorstores import FAISS
from langchain.chains import RetrievalQA

# Load PDF
loader = PyPDFLoader("ml_notes.pdf")
docs = loader.load()

# Split Text
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)

chunks = splitter.split_documents(docs)

# Embeddings
embeddings = GoogleGenerativeAIEmbeddings(
    model="models/embedding-001",
    google_api_key="YOUR_API_KEY"
)

# Vector DB
vectorstore = FAISS.from_documents(
    chunks,
    embeddings
)
```

```python
# Retriever
retriever = vectorstore.as_retriever()

# LLM
llm = ChatGoogleGenerativeAI(
    model="gemini-1.5-flash",
    google_api_key="YOUR_API_KEY"
)

# QA Chain
qa = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever
)

# Query
response = qa.run("Explain Regression")

print(response)
```

---

20. RAG Project Architecture

Components

1. PDF Upload
2. Text Extraction
3. Chunking
4. Embedding Creation
5. Vector Database
6. Retriever
7. LLM Response
8. Chat UI

---

21. Agents in LangChain

Agents allow LLMs to make decisions dynamically.

Agents can:

- Use tools
- Search internet
- Call APIs

- Perform calculations

Agent Workflow

1. User Input
2. Agent Thinking
3. Tool Selection
4. Tool Execution
5. Final Response

Simple Agent Example

```
from langchain.agents import initialize_agent
from langchain.agents import Tool
from langchain_openai import OpenAI

llm = OpenAI(api_key="YOUR_API_KEY")


def calculator_tool(query):
    return eval(query)


tools = [
    Tool(
        name="Calculator",
        func=calculator_tool,
        description="Performs calculations"
    )
]

agent = initialize_agent(
    tools,
    llm,
    agent="zero-shot-react-description",
    verbose=True
)

response = agent.run("What is 50 * 10?")

print(response)
```

---

22. Tools in LangChain

Tools are functions that agents can use.

Examples:

- Calculator
- Search Tool
- Weather API
- Database Query Tool

Custom Tool Example

```
from langchain.tools import tool

@tool
def multiply(a: int, b: int):
    """Multiply two numbers"""
    return a * b

print(multiply.invoke({"a": 5, "b": 10}))
```

---

23. LangChain with OpenAI

```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-4o-mini",
    api_key="YOUR_API_KEY"
)
```

---

24. LangChain with Gemini

```
from langchain_google_genai import ChatGoogleGenerativeAI

model = ChatGoogleGenerativeAI(
```

```
    model="gemini-1.5-flash",
    google_api_key="YOUR_API_KEY"
)
```

---

25. LangChain Expression Language (LCEL)

LCEL is the modern chaining system.

LCEL Example

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI

prompt = ChatPromptTemplate.from_template(
    "Explain {topic}"
)

model = ChatOpenAI(api_key="YOUR_API_KEY")

parser = StrOutputParser()

chain = prompt | model | parser

response = chain.invoke(
    {"topic": "LangChain"}
)

print(response)
```

---

26. Streaming Responses

```
for chunk in chain.stream({"topic": "AI"}):
    print(chunk)
```

---

27. LangServe

LangServe helps deploy LangChain applications as APIs.

Install LangServe

```
pip install langserve
```

---

28. LangSmith

LangSmith helps monitor and debug LangChain apps.

Features:

- Tracing
- Monitoring
- Evaluation
- Debugging

---

29. Best Practices

1. Use proper chunk sizes
2. Optimize prompts
3. Use caching
4. Monitor token usage
5. Use structured outputs
6. Avoid hallucinations using RAG

---

30. Important Interview Questions

Basic Questions

1. What is LangChain?
2. What is RAG?
3. Difference between LLM and Chat Model?
4. What are embeddings?

5. What is a vector database?

Intermediate Questions

1. Explain LangChain architecture
2. Explain memory types
3. What are agents?
4. What are tools?
5. Explain prompt engineering

Advanced Questions

1. Explain LCEL
2. How does RAG improve LLMs?
3. Difference between retriever and vector database?
4. Explain agent workflows
5. Explain streaming in LangChain

---

31. Real-World LangChain Projects

Beginner Projects

1. PDF Chatbot
2. AI Q&A Bot
3. AI Resume Analyzer
4. AI Notes Generator

Intermediate Projects

1. RAG Chatbot
2. AI Research Assistant
3. AI Code Reviewer
4. AI Medical Assistant

Advanced Projects

1. Multi-Agent AI System
2. Autonomous AI Assistant
3. AI Workflow Automation

4. Enterprise RAG System

Common Imports

```
from langchain_openai import ChatOpenAI
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain.prompts import PromptTemplate
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_community.vectorstores import FAISS
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.memory import ConversationBufferMemory
```

Final Tips

- Learn prompt engineering deeply
- Build RAG applications
- Practice agents and tools
- Understand vector databases
- Build real-world projects
- Deploy LangChain applications
- Learn LangGraph next

End of Notes